SIMATS ENGINEERING

ASSESSMENT TOOL 1

COURSE NAME: **Database Management Systems**

COURSE CODE: **CSA05**

COURSE OUTCOME COVERED

CO2: *Apply the relational model, relational algebra, SQL query structures, and views to solve database querying and data manipulation problems.* (BL3, BL6)

Activity Tool : Analytical Problem Solving (High)

Assessment Tool Weightage: 40%

QUESTIONS		
Q.No	**Question**	**Bloom's Level**
1	Given a set of relations and sample data, write SQL queries to retrieve specific information using joins and subqueries, and explain the logic used. Bloom's Level: BL3 – Apply	BL4 – Analyze
2	Using relational algebra, formulate expressions to solve complex queries such as retrieving records that satisfy multiple conditions across relations. Bloom's Level: BL4 – Analyse	BL5 – Evaluate
3	Given a real-world scenario (e.g., banking or student system), design a relational schema and construct appropriate SQL queries for data retrieval and updates. Bloom's Level: BL6 – Create	BL6 – Create
4	Analyse a given SQL query with nested subqueries and predict its output, explaining each step of execution. Bloom's Level: BL4 – Analyze	BL6 – Create

5	Compare different SQL approaches (joins vs subqueries) for solving the same problem and evaluate their efficiency. Bloom's Level: BL5 – Evaluate	BL4 – Analyze
6	Design and implement views for a database system to provide restricted data access, and justify their use for security and abstraction. Bloom's Level: BL6 – Create	BL5 – Evaluate
7	Given a database schema, write relational algebra expressions and convert them into equivalent SQL queries, analyzing differences in representation. Bloom's Level: BL4 – Analyze	BL4 – Analyze
8	Optimize a set of inefficient SQL queries by restructuring them using joins, indexing concepts, or views, and evaluate performance improvements. Bloom's Level: BL5 – Evaluate	BL5 – Evaluate
9	Develop a complex SQL query involving grouping, aggregation, and nested queries to solve a real-world problem. Bloom's Level: BL6 – Create	BL6 – Create
10	Given multiple relations, analyze the query requirements and decide whether to use views, joins, or subqueries, justifying your choice with reasoning. Bloom's Level: BL5 – Evaluate	BL5 – Evaluate

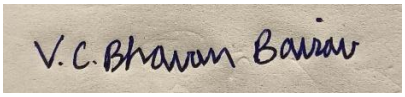
---

Code of Conduct:

I V.C.BHAVAN BAIRAV - 192511369 *(Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.*

I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student:

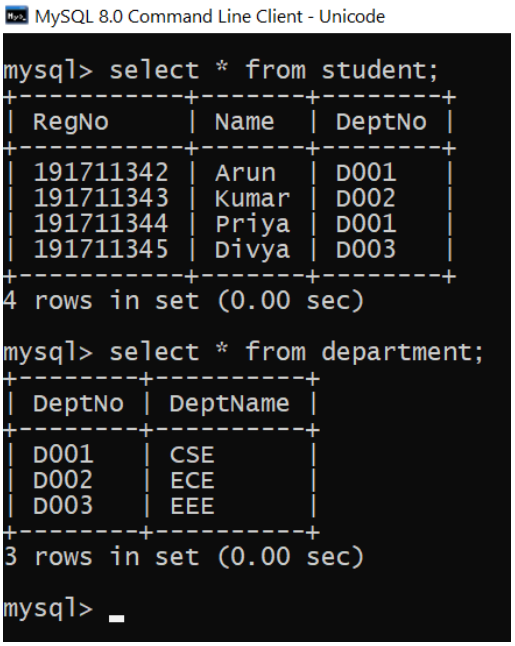


RUBRICS FOR CALCULATION:

Criteria	**Level 4 (Exemplary)**	**Level 3 (Proficient)**	**Level 2 (Developing)**	**Level 1 (Beginning)**
Problem Understanding	Clearly interprets problem, identifies all parameters and requirements accurately	Understands problem with minor gaps	Partial understanding; misses key elements	Misinterprets or unable to understand problem
Method Selection	Selects most appropriate method/formula with strong justification	Selects correct method with minor errors	Inappropriate or partially correct method	Unable to select method
Solution Process	Logical, systematic steps with correct approach throughout	Mostly correct steps with minor mistakes	Disorganized steps; multiple errors	No clear process
Accuracy of Calculation	All calculations accurate; correct final answer	Minor calculation errors	Frequent errors; incorrect final answer	No valid calculations
Interpretation of Results	Clearly interprets results with units and engineering relevance	Basic interpretation with minor omissions	Limited or unclear interpretation	No interpretation

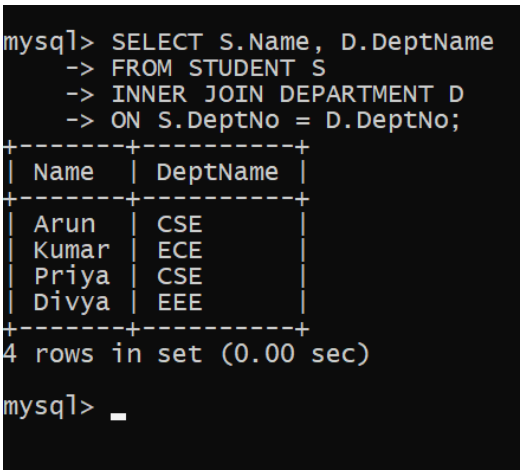
1) Given a set of relations and sample data, write SQL queries to retrieve specific information using joins and subqueries, and explain the logic used.

STUDENT AND DEPARTMENT TABLE:



1. INNER JOIN

OUTPUT:



2. LEFT JOIN

OUTPUT:

```
mysql> SELECT S.Name, D.DeptName
    -> FROM STUDENT S
    -> LEFT JOIN DEPARTMENT D
    -> ON S.DeptNo = D.DeptNo;
+-------+----------+
| Name  | DeptName |
+-------+----------+
| Arun  | CSE      |
| Kumar | ECE      |
| Priya | CSE      |
| Divya | EEE      |
+-------+----------+
4 rows in set (0.00 sec)

mysql>
```

3. RIGHT JOIN

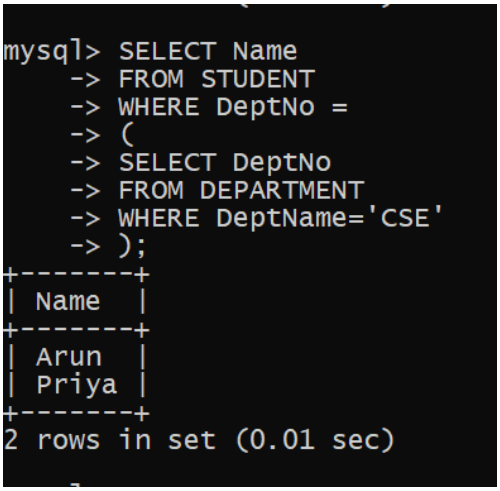
OUTPUT:

```
mysql> SELECT S.Name, D.DeptName
    -> FROM STUDENT S
    -> RIGHT JOIN DEPARTMENT D
    -> ON S.DeptNo = D.DeptNo;
+-------+----------+
| Name  | DeptName |
+-------+----------+
| Priya | CSE      |
| Arun  | CSE      |
| Kumar | ECE      |
| Divya | EEE      |
+-------+----------+
4 rows in set (0.00 sec)

mysql>
```

4. SUBQUERY

Find students whose department is CSE:

OUTPUT:

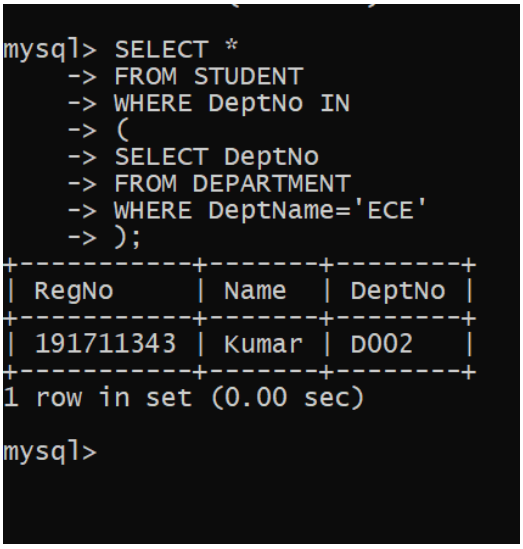


```
mysql> SELECT Name
    -> FROM STUDENT
    -> WHERE DeptNo =
    -> (
    -> SELECT DeptNo
    -> FROM DEPARTMENT
    -> WHERE DeptName='CSE'
    -> );
+-------+
| Name  |
+-------+
| Arun  |
| Priya |
+-------+
2 rows in set (0.01 sec)
```

5. SUBQUERY

Find students whose department is ECE:

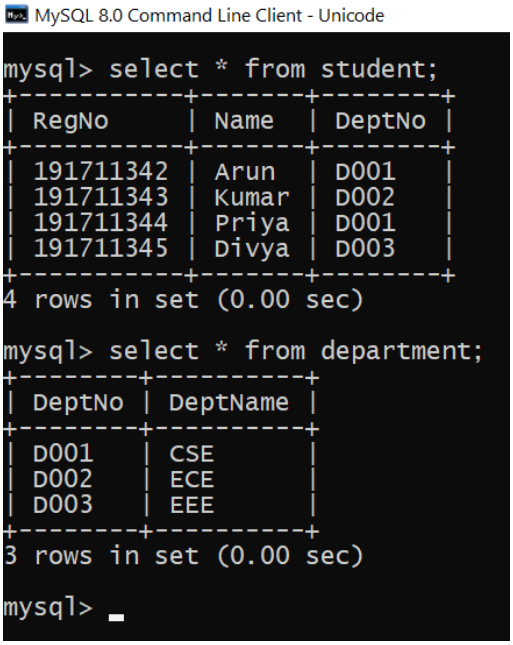
OUTPUT:



```
mysql> SELECT *
    -> FROM STUDENT
    -> WHERE DeptNo IN
    -> (
    -> SELECT DeptNo
    -> FROM DEPARTMENT
    -> WHERE DeptName='ECE'
    -> );
+-----------+-------+--------+
| RegNo     | Name  | DeptNo |
+-----------+-------+--------+
| 191711343 | Kumar | D002   |
+-----------+-------+--------+
1 row in set (0.00 sec)

mysql>
```

2) Using relational algebra, formulate expressions to solve complex queries such as retrieving records that satisfy multiple conditions across relations.

RELATIONS:



RETRIEVE STUDENTS FROM THE CSE DEPARTMENT

QUERY:

π Name (σ DeptName='CSE'

(STUDENT ⋈ STUDENT.DeptNo = DEPARTMENT.DeptNo DEPARTMENT))

RETRIEVE STUDENT NAMES AND DEPARTMENT NAMES

QUERY:

π Name, DeptName

(STUDENT $\bowtie$ STUDENT.DeptNo = DEPARTMENT.DeptNo DEPARTMENT)

RETRIEVE STUDENTS WHOSE DEPTNO IS D001

QUERY:

π Name

(σ DeptNo='D001' (STUDENT))

RETRIEVE ALL DEPARTMENT NAMES

QUERY:

π DeptName (DEPARTMENT)

RETRIEVE ALL FEMALE STUDENTS FROM THE CSE DEPARTMENT

QUERY:

π Name

(σ Gender='F' $\wedge$ DeptName='CSE'

(STUDENT $\bowtie$ STUDENT.DeptNo = DEPARTMENT.DeptNo DEPARTMENT))

3) Given a real-world scenario (e.g., banking or student system), design a relational schema and construct appropriate SQL queries for data retrieval and updates.

CREATION AND INSERTION:

OUTPUT:

```
Copyright (c) 2000, 2026, Oracle and/or its affiliates.

Oracle is a registered trademark of Oracle Corporation and/or its
affiliates. Other names may be trademarks of their respective
owners.

Type 'help;' or '\h' for help. Type '\c' to clear the current input statement.

mysql> create database cry
    -> ;
Query OK, 1 row affected (0.01 sec)

mysql> use cry
Database changed
mysql> ^C
mysql> CREATE TABLE Student (
    ->     StudentID INT PRIMARY KEY,
    ->     StudentName VARCHAR(50),
    ->     Department VARCHAR(50),
    ->     YearOfStudy INT
    -> );
Query OK, 0 rows affected (0.03 sec)

mysql> CREATE TABLE Course (
    ->     CourseID INT PRIMARY KEY,
    ->     CourseName VARCHAR(50),
    ->     Credits INT
    -> );
Query OK, 0 rows affected (0.03 sec)

mysql> CREATE TABLE Enrollment (
    ->     EnrollmentID INT PRIMARY KEY,
    ->     StudentID INT,
    ->     CourseID INT,
    ->     Grade CHAR(2),
    ->     FOREIGN KEY (StudentID) REFERENCES Student(StudentID),
    ->     FOREIGN KEY (CourseID) REFERENCES Course(CourseID)
    -> );
Query OK, 0 rows affected (0.08 sec)
```

```
mysql> INSERT INTO Student VALUES
    -> (101, 'Aryan', 'Computer Science', 2),
    -> (102, 'Rahul', 'Information Technology', 3),
    -> (103, 'Priya', 'Computer Science', 1);
Query OK, 3 rows affected (0.01 sec)
Records: 3  Duplicates: 0  Warnings: 0

mysql> INSERT INTO Course VALUES
    -> (201, 'DBMS', 4),
    -> (202, 'Python Programming', 3),
    -> (203, 'Operating Systems', 4);
Query OK, 3 rows affected (0.01 sec)
Records: 3  Duplicates: 0  Warnings: 0

mysql> INSERT INTO Enrollment VALUES
    -> (1, 101, 201, 'A'),
    -> (2, 101, 202, 'B'),
    -> (3, 102, 201, 'A'),
    -> (4, 103, 203, 'B');
Query OK, 4 rows affected (0.01 sec)
Records: 4  Duplicates: 0  Warnings: 0

mysql> SELECT S.StudentName, C.CourseName
    -> FROM Student S
    -> JOIN Enrollment E ON S.StudentID = E.StudentID
    -> JOIN Course C ON E.CourseID = C.CourseID;
+-------------+--------------------+
| StudentName | CourseName         |
+-------------+--------------------+
| Aryan       | DBMS               |
| Aryan       | Python Programming |
| Rahul       | DBMS               |
| Priya       | Operating Systems  |
+-------------+--------------------+
4 rows in set (0.00 sec)
```

DATA RETRIEVAL

OUTPUT:

```
mysql> SELECT *
    -> FROM Student
    -> WHERE Department = 'Computer Science';
+-----------+-------------+------------------+-------------+
| StudentID | StudentName | Department       | YearOfStudy |
+-----------+-------------+------------------+-------------+
|       101 | Aryan       | Computer Science |           2 |
|       103 | Priya       | Computer Science |           1 |
+-----------+-------------+------------------+-------------+
2 rows in set (0.00 sec)

mysql> SELECT S.StudentName, C.CourseName
    -> FROM Student S
    -> JOIN Enrollment E ON S.StudentID = E.StudentID
    -> JOIN Course C ON E.CourseID = C.CourseID
    -> WHERE E.Grade = 'A';
+-------------+------------+
| StudentName | CourseName |
+-------------+------------+
| Aryan       | DBMS       |
| Rahul       | DBMS       |
+-------------+------------+
2 rows in set (0.00 sec)

mysql> SELECT C.CourseName, COUNT(E.StudentID) AS TotalStudents
    -> FROM Course C
    -> LEFT JOIN Enrollment E
    -> ON C.CourseID = E.CourseID
    -> GROUP BY C.CourseName;
+--------------------+---------------+
| CourseName         | TotalStudents |
+--------------------+---------------+
| DBMS               |             2 |
| Python Programming |             1 |
| Operating Systems  |             1 |
+--------------------+---------------+
3 rows in set (0.00 sec)
```

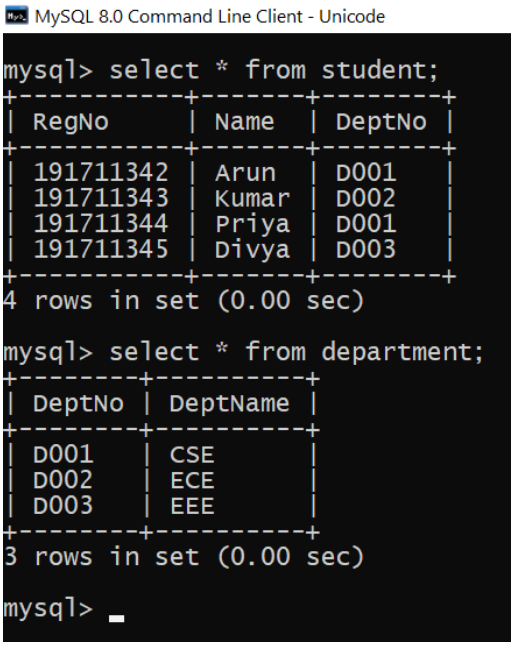
DATA UPDATES:

OUTPUT:

```
mysql> UPDATE Student
    -> SET Department = 'Artificial Intelligence'
    -> WHERE StudentID = 101;
Query OK, 1 row affected (0.01 sec)
Rows matched: 1  Changed: 1  Warnings: 0

mysql> UPDATE Enrollment
    -> SET Grade = 'A'
    -> WHERE StudentID = 103
    -> AND CourseID = 203;
Query OK, 1 row affected (0.01 sec)
Rows matched: 1  Changed: 1  Warnings: 0

mysql> INSERT INTO Enrollment
    -> VALUES (5, 102, 202, 'B');
Query OK, 1 row affected (0.00 sec)

mysql> DELETE FROM Enrollment
    -> WHERE EnrollmentID = 5;
Query OK, 1 row affected (0.01 sec)

mysql>
mysql> select * from enrollment
    -> ;
+--------------+-----------+----------+-------+
| EnrollmentID | StudentID | CourseID | Grade |
+--------------+-----------+----------+-------+
|            1 |       101 |      201 | A     |
|            2 |       101 |      202 | B     |
|            3 |       102 |      201 | A     |
|            4 |       103 |      203 | A     |
+--------------+-----------+----------+-------+
4 rows in set (0.00 sec)
```

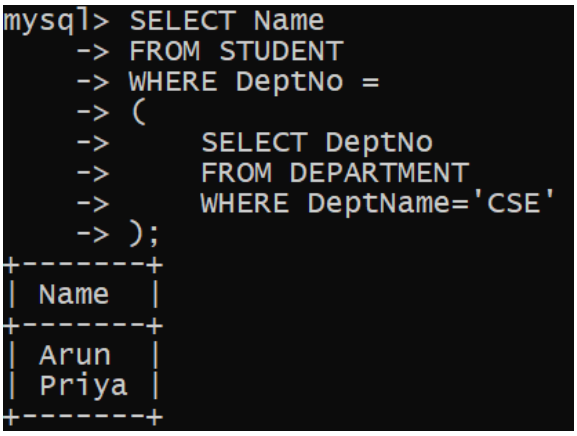
4) Analyse a given SQL query with nested subqueries and predict its output, explaining each step of execution.

STUDENT AND DEPARTMENT TABLE



SQL QUERY WITH NESTED SUBQUERY

OUTPUT:



Step-by-Step Execution

Step 1: Inner Query Executes

```
mysql> SELECT DeptNo
    -> FROM DEPARTMENT
    -> WHERE DeptName='CSE';
+--------+
| DeptNo |
+--------+
| D001   |
+--------+
1 row in set (0.00 sec)
```

Step 2: Outer Query Executes

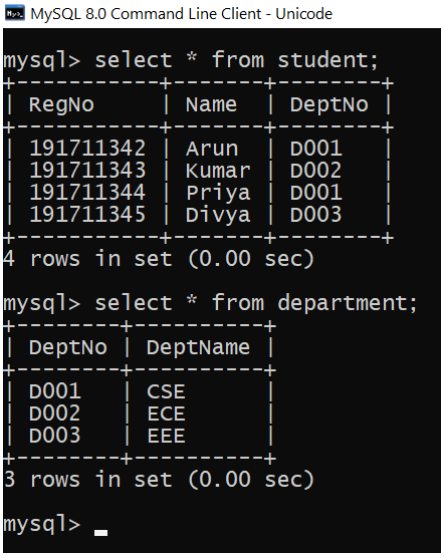
```
mysql> SELECT Name
    -> FROM STUDENT
    -> WHERE DeptNo='D001';
+-------+
| Name  |
+-------+
| Arun  |
| Priya |
+-------+
2 rows in set (0.00 sec)
```

Final Output:

```
+-------+
| Name  |
+-------+
| Arun  |
| Priya |
+-------+
```

5) Compare different SQL approaches (joins vs subqueries) for solving the same problem and evaluate their efficiency.

STUDENT AND DEPARTMENT TABLE



METHOD 1: USING JOIN

OUTPUT:

```
mysql> use c02;
Database changed
mysql> SELECT S.Name, D.DeptName
    -> FROM STUDENT S
    -> JOIN DEPARTMENT D
    -> ON S.DeptNo = D.DeptNo
    -> WHERE D.DeptName = 'CSE';
+-------+----------+
| Name  | DeptName |
+-------+----------+
| Arun  | CSE      |
| Priya | CSE      |
+-------+----------+
2 rows in set (0.01 sec)
```

METHOD 2: USING SUBQUERY

OUTPUT:

```
mysql> SELECT Name
    -> FROM STUDENT
    -> WHERE DeptNo =
    -> (
    ->     SELECT DeptNo
    ->     FROM DEPARTMENT
    ->     WHERE DeptName = 'CSE'
    -> );
+-------+
| Name  |
+-------+
| Arun  |
| Priya |
+-------+
2 rows in set (0.00 sec)
```

6) Design and implement views for a database system to provide restricted data access, and justify their use for security and abstraction.

CREATE AND DISPLAY A VIEW

OUTPUT:

```
mysql> use hello
Database changed
mysql> CREATE TABLE Student (
    ->     StudentID INT PRIMARY KEY,
    ->     StudentName VARCHAR(50),
    ->     Department VARCHAR(50),
    ->     Email VARCHAR(100),
    ->     GPA DECIMAL(3,2)
    -> );
Query OK, 0 rows affected (0.03 sec)

mysql> INSERT INTO Student VALUES
    -> (101, 'Aryan', 'Computer Science', 'aryan@gmail.com', 8.9),
    -> (102, 'Rahul', 'Information Technology', 'rahul@gmail.com', 8.2),
    -> (103, 'Priya', 'Computer Science', 'priya@gmail.com', 9.1);
Query OK, 3 rows affected (0.01 sec)
Records: 3  Duplicates: 0  Warnings: 0

mysql> CREATE VIEW Student_Public_View AS
    -> SELECT StudentID,
    ->        StudentName,
    ->        Department
    -> FROM Student;
Query OK, 0 rows affected (0.01 sec)

mysql> SELECT * FROM Student_Public_View;
+-----------+-------------+-----------------------+
| StudentID | StudentName | Department            |
+-----------+-------------+-----------------------+
|       101 | Aryan       | Computer Science      |
|       102 | Rahul       | Information Technology |
|       103 | Priya       | Computer Science      |
+-----------+-------------+-----------------------+
3 rows in set (0.00 sec)
```

CREATE ANOTHER VIEW WITH CONDITION

OUTPUT:

```
mysql> CREATE VIEW CS_Students_View AS
    -> SELECT StudentID,
    ->        StudentName,
    ->        GPA
    -> FROM Student
    -> WHERE Department = 'Computer Science';
Query OK, 0 rows affected (0.01 sec)

mysql> SELECT * FROM CS_Students_View;
+-----------+-------------+------+
| StudentID | StudentName | GPA  |
+-----------+-------------+------+
|       101 | Aryan       | 8.90 |
|       103 | Priya       | 9.10 |
+-----------+-------------+------+
2 rows in set (0.00 sec)
```

UPDATE THROUGH VIEW

OUTPUT:

```
mysql> UPDATE Student_Public_View
    -> SET Department='AI & DS'
    -> WHERE StudentID=101;
Query OK, 1 row affected (0.01 sec)
Rows matched: 1  Changed: 1  Warnings: 0

mysql> SELECT * FROM Student;
+-----------+-------------+-------------------------+-----------------+------+
| StudentID | StudentName | Department              | Email           | GPA  |
+-----------+-------------+-------------------------+-----------------+------+
|       101 | Aryan       | AI & DS                 | aryan@gmail.com | 8.90 |
|       102 | Rahul       | Information Technology  | rahul@gmail.com | 8.20 |
|       103 | Priya       | Computer Science        | priya@gmail.com | 9.10 |
+-----------+-------------+-------------------------+-----------------+------+
3 rows in set (0.00 sec)
```

INSERT THROUGH VIEW

OUTPUT:

```
mysql> INSERT INTO Student_Public_View
    -> VALUES
    -> (104,'Karthik','Electronics');
Query OK, 1 row affected (0.01 sec)

mysql> SELECT * FROM Student;
+-----------+-------------+------------------------+-----------------+------+
| StudentID | StudentName | Department             | Email           | GPA  |
+-----------+-------------+------------------------+-----------------+------+
|       101 | Aryan       | AI & DS                | aryan@gmail.com | 8.90 |
|       102 | Rahul       | Information Technology | rahul@gmail.com | 8.20 |
|       103 | Priya       | Computer Science       | priya@gmail.com | 9.10 |
|       104 | Karthik     | Electronics            | NULL            | NULL |
+-----------+-------------+------------------------+-----------------+------+
4 rows in set (0.00 sec)
```

DELETE THROUGH VIEW

OUTPUT:

```
mysql> DELETE FROM Student_Public_View
    -> WHERE StudentID=104;
Query OK, 1 row affected (0.01 sec)

mysql> SELECT * FROM Student;
+-----------+-------------+------------------------+-----------------+------+
| StudentID | StudentName | Department             | Email           | GPA  |
+-----------+-------------+------------------------+-----------------+------+
|       101 | Aryan       | AI & DS                | aryan@gmail.com | 8.90 |
|       102 | Rahul       | Information Technology | rahul@gmail.com | 8.20 |
|       103 | Priya       | Computer Science       | priya@gmail.com | 9.10 |
+-----------+-------------+------------------------+-----------------+------+
3 rows in set (0.00 sec)
```

REPLACE VIEW

OUTPUT:

```
mysql> CREATE OR REPLACE VIEW Student_Public_View AS
    -> SELECT StudentID, StudentName, Department, GPA
    -> FROM Student;
Query OK, 0 rows affected (0.01 sec)

mysql> SELECT * FROM Student_Public_View;
+-----------+-------------+-----------------------+------+
| StudentID | StudentName | Department            | GPA  |
+-----------+-------------+-----------------------+------+
|       101 | Aryan       | AI & DS               | 8.90 |
|       102 | Rahul       | Information Technology | 8.20 |
|       103 | Priya       | Computer Science      | 9.10 |
+-----------+-------------+-----------------------+------+
3 rows in set (0.00 sec)
```

DESCRIBE VIEW

OUTPUT:

```
mysql> DESC Student_Public_View;
+-------------+--------------+------+-----+---------+-------+
| Field       | Type         | Null | Key | Default | Extra |
+-------------+--------------+------+-----+---------+-------+
| StudentID   | int          | NO   |     | NULL    |       |
| StudentName | varchar(50)  | YES  |     | NULL    |       |
| Department  | varchar(50)  | YES  |     | NULL    |       |
| GPA         | decimal(3,2) | YES  |     | NULL    |       |
+-------------+--------------+------+-----+---------+-------+
4 rows in set (0.00 sec)
```

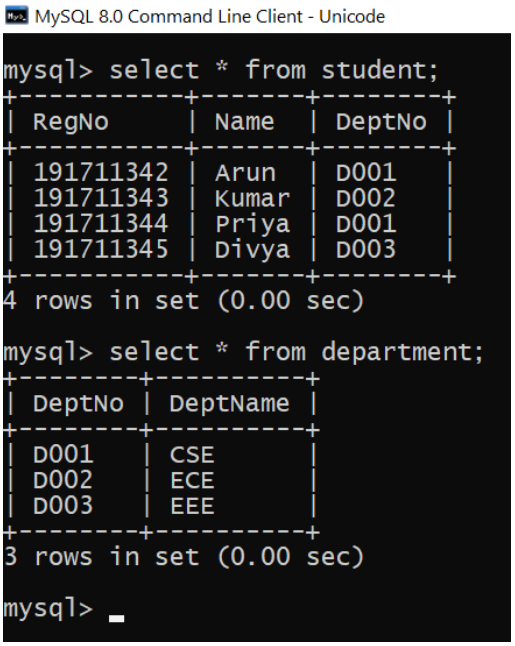
DROP VIEW

OUTPUT:

```
mysql> DROP VIEW Student_Public_View;
Query OK, 0 rows affected (0.01 sec)
```

7) Given a database schema, write relational algebra expressions and convert them into equivalent SQL queries, analyzing differences in representation.

STUDENT AND DEPARTMENT TABLE



DISPLAY STUDENT NAMES

Relational Algebra:

π Name (STUDENT)

Equivalent SQL:

```
mysql> SELECT Name
    -> FROM STUDENT;
+-------+
| Name  |
+-------+
| Arun  |
| Kumar |
| Priya |
| Divya |
+-------+
4 rows in set (0.00 sec)
```

DISPLAY STUDENTS FROM CSE DEPARTMENT

Relational Algebra:

π Name

(σ DeptName='CSE'

(STUDENT ⋈ STUDENT.DeptNo = DEPARTMENT.DeptNo DEPARTMENT))

Equivalent SQL:

```
mysql> SELECT S.Name
    -> FROM STUDENT S
    -> JOIN DEPARTMENT D
    -> ON S.DeptNo = D.DeptNo
    -> WHERE D.DeptName='CSE';
+-------+
| Name  |
+-------+
| Arun  |
| Priya |
+-------+
2 rows in set (0.00 sec)
```

DISPLAY STUDENT NAME AND DEPARTMENT NAME

Relational Algebra:

π Name, DeptName

(STUDENT ⋈ STUDENT.DeptNo = DEPARTMENT.DeptNo DEPARTMENT)

Equivalent SQL:

```
mysql> SELECT S.Name, D.DeptName
    -> FROM STUDENT S
    -> JOIN DEPARTMENT D
    -> ON S.DeptNo = D.DeptNo;
+-------+----------+
| Name  | DeptName |
+-------+----------+
| Arun  | CSE      |
| Kumar | ECE      |
| Priya | CSE      |
| Divya | EEE      |
+-------+----------+
4 rows in set (0.00 sec)
```

DISPLAY STUDENTS FROM DEPARTMENT D001

Relational Algebra:

π Name

(σ DeptNo='D001' (STUDENT))

Equivalent SQL:

```
mysql> SELECT Name
    -> FROM STUDENT
    -> WHERE DeptNo='D001';
+-------+
| Name  |
+-------+
| Arun  |
| Priya |
+-------+
2 rows in set (0.00 sec)
```

8) Optimize a set of inefficient SQL queries by restructuring them using joins, indexing concepts, or views, and evaluate performance improvements.

STUDENT AND DEPARTMENT TABLE

```
mysql> SELECT * FROM STUDENT;
+-----------+-------------+-----------------------+-----------------+------+
| StudentID | StudentName | Department            | Email           | GPA  |
+-----------+-------------+-----------------------+-----------------+------+
|       101 | Aryan       | Computer Science      | aryan@gmail.com | 8.90 |
|       102 | Rahul       | Information Technology | rahul@gmail.com | 8.20 |
|       103 | Priya       | Computer Science      | priya@gmail.com | 9.10 |
+-----------+-------------+-----------------------+-----------------+------+
3 rows in set (0.00 sec)

mysql> SELECT * FROM DEPARTMENT;
+-----------------------+-----------+
| Department            | HOD       |
+-----------------------+-----------+
| Computer Science      | Dr. Kumar |
| Information Technology | Dr. Meena |
| Electronics           | Dr. Raj   |
+-----------------------+-----------+
3 rows in set (0.00 sec)
```

OPTIMIZING USING JOIN

Inefficient Query:

```
mysql> SELECT StudentName
    -> FROM Student
    -> WHERE Department IN
    -> (
    ->     SELECT Department
    ->     FROM Department
    ->     WHERE HOD='Dr. Kumar'
    -> );
+-------------+
| StudentName |
+-------------+
| Aryan       |
| Priya       |
+-------------+
2 rows in set (0.00 sec)
```

Optimized Query (Using JOIN):

```
mysql> SELECT S.StudentName
    -> FROM Student S
    -> JOIN Department D
    -> ON S.Department = D.Department
    -> WHERE D.HOD='Dr. Kumar';
+-------------+
| StudentName |
+-------------+
| Aryan       |
| Priya       |
+-------------+
2 rows in set (0.00 sec)
```

Performance Improvement:

- JOIN executes in a single operation.
- Faster for large datasets.
- Easier to read and maintain.

OPTIMIZING USING INDEX

Inefficient Query:

```
mysql> SELECT *
    -> FROM Student
    -> WHERE Department='Computer Science';
+-----------+-------------+------------------+-----------------+------+
| StudentID | StudentName | Department       | Email           | GPA  |
+-----------+-------------+------------------+-----------------+------+
|       101 | Aryan       | Computer Science | aryan@gmail.com | 8.90 |
|       103 | Priya       | Computer Science | priya@gmail.com | 9.10 |
+-----------+-------------+------------------+-----------------+------+
2 rows in set (0.00 sec)
```

Optimized Query (Using INDEX):

```
mysql> CREATE INDEX idx_department
    -> ON Student(Department);
Query OK, 0 rows affected (0.06 sec)
Records: 0  Duplicates: 0  Warnings: 0

mysql> SELECT *
    -> FROM Student
    -> WHERE Department='Computer Science';
+-----------+-------------+------------------+-----------------+------+
| StudentID | StudentName | Department       | Email           | GPA  |
+-----------+-------------+------------------+-----------------+------+
|       101 | Aryan       | Computer Science | aryan@gmail.com | 8.90 |
|       103 | Priya       | Computer Science | priya@gmail.com | 9.10 |
+-----------+-------------+------------------+-----------------+------+
2 rows in set (0.00 sec)
```

Performance Improvement:

- Faster searching on the Department column.
- Reduces full table scan.
- Improves query execution speed.

OPTIMIZING USING VIEW

Inefficient Query:

```
mysql> SELECT StudentID,
    ->        StudentName,
    ->        Department,
    ->        GPA
    -> FROM Student
    -> WHERE Department='Computer Science';
+-----------+-------------+------------------+------+
| StudentID | StudentName | Department       | GPA  |
+-----------+-------------+------------------+------+
|       101 | Aryan       | Computer Science | 8.90 |
|       103 | Priya       | Computer Science | 9.10 |
+-----------+-------------+------------------+------+
2 rows in set (0.00 sec)
```

Optimized Query (Using VIEW):

```
mysql> CREATE VIEW CS_Students AS
    -> SELECT StudentID,
    ->        StudentName,
    ->        Department,
    ->        GPA
    -> FROM Student
    -> WHERE Department='Computer Science';
Query OK, 0 rows affected (0.01 sec)

mysql> SELECT *
    -> FROM CS_Students;
+-----------+-------------+------------------+------+
| StudentID | StudentName | Department       | GPA  |
+-----------+-------------+------------------+------+
|       101 | Aryan       | Computer Science | 8.90 |
|       103 | Priya       | Computer Science | 9.10 |
+-----------+-------------+------------------+------+
2 rows in set (0.00 sec)
```

Performance Improvement:

- Eliminates writing the same query repeatedly.
- Improves readability.
- Provides abstraction and restricted data access.
- Easier to maintain and reuse.

9) Develop a complex SQL query involving grouping, aggregation, and nested queries to solve a real-world problem.

STUDENT TABLE

```
mysql> SELECT * FROM STUDENT;
+-----------+-------------+-----------------------+-------------------+------+
| StudentID | StudentName | Department            | Email             | GPA  |
+-----------+-------------+-----------------------+-------------------+------+
|       101 | Aryan       | Computer Science      | aryan@gmail.com   | 8.90 |
|       102 | Rahul       | Information Technology | rahul@gmail.com   | 8.20 |
|       103 | Priya       | Computer Science      | priya@gmail.com   | 9.10 |
|       104 | Karthik     | Electronics           | karthik@gmail.com | 7.80 |
|       105 | Neha        | Computer Science      | neha@gmail.com    | 8.70 |
|       106 | Riya        | Information Technology | riya@gmail.com    | 9.00 |
+-----------+-------------+-----------------------+-------------------+------+
6 rows in set (0.00 sec)
```

OUTPUT

```
mysql> SELECT Department,
    ->        AVG(GPA) AS Average_GPA
    -> FROM Student
    -> GROUP BY Department
    -> HAVING AVG(GPA) >
    -> (
    ->     SELECT AVG(GPA)
    ->     FROM Student
    -> );
+------------------+-------------+
| Department       | Average_GPA |
+------------------+-------------+
| Computer Science |    8.900000 |
+------------------+-------------+
1 row in set (0.00 sec)
```

10) Given multiple relations, analyze the query requirements and decide whether to use views, joins, or subqueries, justifying your choice with reasoning.

STUDENT AND DEPARTMENT TABLE

```
mysql> select * from student;
+-----------+-------------+-----------------------+-----------------+------+
| StudentID | StudentName | Department            | Email           | GPA  |
+-----------+-------------+-----------------------+-----------------+------+
|       101 | Aryan       | Computer Science      | aryan@gmail.com | 8.90 |
|       102 | Rahul       | Information Technology | rahul@gmail.com | 8.20 |
|       103 | Priya       | Computer Science      | priya@gmail.com | 9.10 |
+-----------+-------------+-----------------------+-----------------+------+
3 rows in set (0.00 sec)

mysql> select * from department
    -> ;
+-----------------------+-----------+
| Department            | HOD       |
+-----------------------+-----------+
| Computer Science      | Dr. Kumar |
| Information Technology | Dr. Meena |
+-----------------------+-----------+
2 rows in set (0.00 sec)
```

JOIN

Requirement:

Display each student's name along with the HOD of their department.

Output:

```
mysql> SELECT S.StudentName,
    ->        D.HOD
    -> FROM Student S
    -> JOIN Department D
    -> ON S.Department = D.Department;
+-------------+-----------+
| StudentName | HOD       |
+-------------+-----------+
| Aryan       | Dr. Kumar |
| Rahul       | Dr. Meena |
| Priya       | Dr. Kumar |
+-------------+-----------+
3 rows in set (0.00 sec)
```

Justification:

- Two related tables are involved.
- JOIN retrieves data efficiently from both tables.
- Best choice for combining related data.

SUBQUERY

Requirement:

Display students whose GPA is greater than the average GPA.

Output:

```
mysql> SELECT StudentName, GPA
    -> FROM Student
    -> WHERE GPA >
    -> (
    ->     SELECT AVG(GPA)
    ->     FROM Student
    -> );
+-------------+------+
| StudentName | GPA  |
+-------------+------+
| Aryan       | 8.90 |
| Priya       | 9.10 |
+-------------+------+
2 rows in set (0.00 sec)
```

Justification:

- The query compares each student's GPA with an aggregate value.
- A subquery is the simplest and most suitable approach.

VIEW

Requirement:

Frequently display only Computer Science students.

Output:

```
mysql> CREATE VIEW CS_Students AS
    -> SELECT StudentID,
    ->        StudentName,
    ->        GPA
    -> FROM Student
    -> WHERE Department='Computer Science';
Query OK, 0 rows affected (0.01 sec)

mysql> SELECT * FROM CS_Students;
+-----------+-------------+------+
| StudentID | StudentName | GPA  |
+-----------+-------------+------+
|       101 | Aryan       | 8.90 |
|       103 | Priya       | 9.10 |
+-----------+-------------+------+
2 rows in set (0.00 sec)
```

Justification:

- The same query is executed repeatedly.
- A view simplifies the query.
- Improves security by exposing only required columns.